

1. Overview:

The previous week's transaction blueprint outlined the input details (sender, recipient, amount, timestamp, signature), omitted the mechanism which may be applied to verify the party who authorised the transaction. This assignment fulfils this requirement by assigning an ECDSA (SECP256k1) key pair, wrapped in a wallet class, to each transacting party. An address for the wallet is derived from a transacting party's public key. The ability to sign a canonical transaction payload is demonstrated. This ability is integral to verification since it enables other parties to use the sender's unique public key to verify that the information has not been tampered with and that the sender authorised the exact information included.

2. Design:

2.1 Library and curve:

The `ecdsa` package is employed on the SECP256k1 curve, a curve also employed by bitcoin. Signing and verification are facilitated via `SigningKey.sign()` and `VerifyingKey.verify()`.

2.2 Deriving the wallet address:

In the aim of deriving a simplified wallet address from a shortened public key, the following steps are taken:

Step	Python function:	Intention:	Purpose:
1	<code>def pubkey_to_address(pubkey_bytes: bytes)</code>	Represent the public key as raw bytes	Enables conversion of public key into a simplified address for the wallet.
2	<code>->str</code>	The function then returns a string	Strings are important because they preserve the exact alphanumeric characters and sequence of the wallet's address without risking data loss or mathematical distortion.
3	<code>hashlib.sha256(pubkey_bytes)</code>	The public key is hashed using SHA256, which produces a 256-bit (32-byte) hash.	The hash is useful since it generates a fixed-length identifier regardless of the original public key length.
4	<code>.hexdigest()</code>	Converts the hash into a hexadecimal	The 32 byte hash generated from SHA256 is represented as 64 hexadecimal characters using hexadecimal
5	<code>[:40]</code>	Simplification of public key for the wallet address	The 64 character SHA256 hash is truncated to 40 hexadecimal characters.

The implemented steps above are not a replication of the P2PKH/Bech32 address format employed by Bitcoin. This approach diverges from that of Bitcoin by skipping RIPEMD-160, Base58Check and the network/version byte completely.

2.3 The canonical payload:

The bytes eligible for signing include elements of the standardised transaction message. Four transaction fields (including the sender, recipient, amount and timestamp) are defined and set up in dictionary format. The dictionary keys are subsequently ordered alphabetically, to ensure uniform representation, with separators applied to remove unnecessary spaces. The JSON string is subsequently converted into bytes, which are signed by the wallet. The signature is purposefully excluded because we are trying to calculate the signature itself. Only the transaction data is signed, with the signature being subsequently attached to the transaction. Excluding the signature is integral for verification, when a party receives the transaction, they remove/ignore the signature, recreate the same canonical payload and verify the signature against it.

3. Execution evidence:

```

== Wallets ==
Phenyo address: c7bfe2bb619a5f0eda956cf9a585e9ec7309fb1e
Thato address: e9a9fcd95cc06607e956be777b4e767b9ff1217b
Distinct addresses: True

== Canonical payload (Phenyo -> Thato, amount=10) ==
{"amount":10.0,"recipient":"e9a9fcd95cc06607e956be777b4e767b9ff1217b","sender":"c7bfe2bb619a5f0eda956cf9a585e9ec7309fb1e","timestamp":1700000000.0}

== Signature ==
signature (hex, truncated): 071bc18dfc63e52afe626f0ebb58ac69...

== Verification checkpoints ==
verify(original payload, Phenyo's key): True
verify(tampered payload, Phenyo's key): False
verify(original payload, Thato's key): False

All checkpoints passed: signing binds Phenyo to these exact bytes,
and any change to the amount or the verifying key breaks that binding.

```

Checkpoint analysis:

Checkpoint:	Aim:	Result:	Interpretation:
1	Baseline case: Verifies whether the signature matches the original transaction data and Phenyo's key	TRUE	Phenyo created the signature using Phenyo's private key and the verifier uses Phenyo's public key to check it.
2	Mutation failure demonstration: A party altered the amount from 10 to 1000 but keeps the original signature.	FALSE	Since the signature was generated using { Phenyo , Thato, 10, 1700000} the bytes change when the input is tampered with. Hence the verification fails when the payload changes, since the signature no longer matches.
3	Ownership demonstration: The original payload is verified against the recipients public key rather than the sender who originated it.	FALSE	The transaction was signed using Phenyo's private key, rather than Thato's. Hence, the signature is only validated under the key of the party who actually produced it.

4. Real-world limitations and risks (Security note):

While the ECDSA signature employed in this demonstration proves that the party holding the private key corresponding to the public key authorised the transaction bytes, it does not prove the off-chain compliance processes necessary for legal and regulatory acceptability.

In addition, the demonstration emphasises the importance of the private key, the signature is only trustworthy if the private key is secure. If an external party gains access to the wallet owner's private key, they could potentially create signatures which verify perfectly, given that the blockchain is unable to distinguish whether the user is the wallet owner or a party who stole the key. This substantiates the classification of key custody as an operation risk, rather than just a cryptographic problem. For a remittance wallet, where transactions occur internationally, private-key custody risks include:

- A digital asset wallet, constantly connected to the internet, storing private keys online, being accessible through a careless operators laptop.
- Sensitive data such as passwords being shared across unauthorised parties
- A lack of secondary verification steps being implemented by the remittance facilitator on large transactions
- Private keys being leaked into application logs, chat back ups or continuous integration artefacts

In reality, certain security measures can be taken. What I would not do:

- Private keys should not be reused across all wallet owners or hard-coded in source.
- Fail to use Hardware Security modules: Hardware security modules should be used to keep cryptographic keys in a protected environment, enabling applications to use the key for signing without exposing the private key itself.
- Keys should not be committed to a public repository or logged/printed outside a local, disposable demo.

5. Reflection:

This assignment shows how cryptography turns a transaction from something we trust into something which is mathematically verifiable. Using the digital signature enables us to verify whether the transaction was actually authorised by the holder of the wallet's private key. This is useful for transactions since the system now has an objective cryptographic test to validate the transaction details. Signing is not encryption, however. The digital signature is used to verify authenticity and integrity but does not guarantee confidentiality. The transaction data can be observed, and the signature enables detection of unauthorised modification, confidentiality would require an additional privacy/encryption mechanism.

Appendix: AI-Assistance Declaration

Tool: Claude (Anthropic), claude.ai

What it was used for: Fixing errors in wallet.py code file, I had to convert the file to a .py and needed assistance;